U03 · Excepciones, bucles, arrays y métodos

Con los tipos, variables y `if`/`switch` de la Unidad 2 ya puedes tomar decisiones. Ahora toca **repetir** trabajo, **almacenar muchos datos a la vez**, **organizar tu código en piezas reutilizables** y **controlar lo que falla** — los cuatro pilares que te van a acompañar el resto del curso.

1. Bucles en Java

`while`: repetir mientras se cumpla una condición

```
while (condición) {
    // se repite mientras condición sea true
}
```

La condición se evalúa **antes** de cada vuelta — igual que en pseudocódigo, cabe la posibilidad de que el cuerpo no se ejecute ni una vez.

Hay dos formas típicas de controlar un `while`, y conviene distinguirlas porque cada una encaja en un tipo de problema distinto:

- **Por contador**: sabes de antemano cuántas veces se repite (un valor inicial, cuánto cambia en cada vuelta, y un valor final).

  ```
  int i = 1;
  while (i <= 100) {
      System.out.println(i);
      i++;   // si olvidas esto, el bucle no termina nunca
  }
  ```

- **Por centinela**: el bucle se repite hasta que se cumple (o deja de cumplirse) una condición que no depende de contar repeticiones, sino de un valor concreto (por ejemplo, hasta que el usuario introduce un texto vacío, o hasta que sale un múltiplo de 7 al azar).

```java
boolean salir = false;
while (!salir) {
    int n = (int) (Math.random() * 500) + 1;
    System.out.println(n);
    salir = (n % 7 == 0);
}
```

> ⚠ **El bucle infinito**
>
> Si te olvidas de actualizar el contador o el centinela dentro del bucle, la condición nunca deja de cumplirse y el programa se queda "colgado" (consumiendo CPU o memoria sin parar, hasta que lo mates a la fuerza). Antes de ejecutar cualquier bucle nuevo, pregúntate: *¿hay alguna sentencia dentro que, tarde o temprano, haga falsa la condición?*

`do-while`: se ejecuta siempre al menos una vez

```java
do {
    // se ejecuta, y LUEGO se comprueba la condición
} while (condición);
```

La única diferencia con `while` es *cuándo* se evalúa la condición: aquí, después de ejecutar el cuerpo — así que el cuerpo se ejecuta siempre al menos una vez, aunque la condición ya fuera falsa desde el principio.

`for`: cuando conoces el número de repeticiones

```java
for (int i = 1; i <= 10; i++) {
    System.out.println(i);
}
```

Las tres partes entre paréntesis son: **inicialización** (se ejecuta una sola vez, al empezar), **condición** (se comprueba antes de cada vuelta, igual que en `while`) e **incremento** (se ejecuta al final de cada vuelta). Es, en el fondo, un `while` con esas tres partes agrupadas y obligatorias de escribir juntas — por eso es la forma preferida cuando el bucle es "por contador".

`break` y `continue`

- **`break`**: corta el bucle inmediatamente, saltando a la primera sentencia después de él.
- **`continue`**: salta directamente a la siguiente iteración, sin ejecutar el resto del cuerpo en la vuelta actual.

```
for (int i = 1; i <= 20; i++) {
    if (i % 2 != 0) continue;   // salta los impares
    if (i > 10) break;          // corta al llegar a un par mayor que 10
    System.out.println(i);
}
```

> ⚠ **El punto y coma solitario**
>
> ```
> for (int i = 0; i < 10; i++);
> System.out.println("Hola");
> ```
>
> Este código imprime "Hola" **una sola vez**, no diez. El `;` justo después del `for` es una **sentencia vacía** ("no hacer nada") — el bucle se repite 10 veces sin hacer nada, y el `println` de verdad está *fuera* del bucle, así que se ejecuta una única vez. Es un error de lectura fácil de cometer y muy difícil de detectar a simple vista — presta siempre atención a dónde acaba realmente cada línea.

2. Arrays

Imagina que necesitas guardar los nombres de 1000 personas. Declarar 1000 variables sueltas (`nombre1, nombre2, ... nombre1000`) sería absurdo. Un **array** es un objeto que guarda **varios valores del mismo tipo**, cada uno en una posición numerada — su **índice**, empezando siempre en `0`.

```java
int[] numeros;                    // 1. declarar: solo la referencia, aún no existe el array
numeros = new int[5];             // 2. crear: reserva espacio para 5 enteros (todos a 0)
numeros[0] = 10;                  // 3. usar: asignar/leer por índice
System.out.println(numeros[0]);

int[] otros = {1, 2, 3, 4, 5};    // declarar + crear + inicializar de golpe, con valores concretos
```

- `array.length` te da su tamaño (es un atributo, sin paréntesis — no confundir con `String.length()`, que sí lleva paréntesis).
- El primer índice válido es `0`, el último es `array.length - 1`. Acceder fuera de ese rango no da un valor raro: lanza una excepción (`ArrayIndexOutOfBoundsException`) y el programa se detiene si no la atrapas.
- Un `for` es la forma natural de recorrer un array entero:

  ```java
  for (int i = 0; i < numeros.length; i++) {
      System.out.println(numeros[i]);
  }
  // forma abreviada, cuando no necesitas el índice:
  for (int n : numeros) {
      System.out.println(n);
  }
  ```

Arrays multidimensionales

Un array puede contener a su vez otros arrays — lo más habitual es una **matriz** (array bidimensional), útil para representar una tabla, un tablero, una cuadrícula:

```java
int[][] tablero = new int[3][3];   // matriz 3x3, todos los valores a 0
tablero[1][2] = 5;                 // fila 1, columna 2

int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6}
};   // matriz simétrica: todas las filas tienen la misma longitud
```

En Java, un array bidimensional es en realidad "un array de arrays" — eso permite crear matrices **asimétricas**, donde cada fila tiene una longitud distinta:

```
int[][] triangular = new int[3][];
triangular[0] = new int[]{1};
triangular[1] = new int[]{1, 2};
triangular[2] = new int[]{1, 2, 3};
```

La clase `Arrays`

El paquete `java.util` incluye la clase `Arrays`, con métodos estáticos que evitan reinventar operaciones muy comunes:

```
import java.util.Arrays;

int[] a = {5, 3, 1, 4, 2};
Arrays.sort(a);                          // ordena el array en el propio sitio (in-
place)
System.out.println(Arrays.toString(a));  // "[1, 2, 3, 4, 5]" — para imprimirlo de
forma legible
int[] copia = Arrays.copyOf(a, a.length); // copia independiente del array
boolean iguales = Arrays.equals(a, copia); // compara contenido, no ==
```

⭐ **Be the Code**: nunca compares dos arrays con `==` (compara si son el mismo objeto en memoria, exactamente el mismo problema que ya viste con `String`) ni los imprimas directamente con `System.out.println(array)` (te imprime algo como `[I@1b6d3586`, la dirección de memoria, no el contenido) — usa siempre `Arrays.equals(...)` y `Arrays.toString(...)`.

3. Procesar `String` con bucles

Un `String` se puede recorrer carácter a carácter, igual que un array:

```
String palabra = "programar";
for (int i = 0; i < palabra.length(); i++) {
    System.out.println(palabra.charAt(i));
}
```

Esto te permite implementar algoritmos clásicos de procesamiento de texto tú mismo: contar vocales, comprobar si una palabra es un palíndromo, buscar la posición de un carácter concreto, invertir una cadena… antes de usar librerías que ya lo hacen por ti (como `StringBuilder.reverse()`), es un ejercicio muy útil para entender de verdad cómo funcionan los bucles combinados con condiciones.

4. Métodos

Ya viste los subalgoritmos en pseudocódigo — en Java se llaman **métodos**, y son la forma de aplicar la idea de "divide y vencerás": descomponer un programa grande en piezas pequeñas, cada una con **una única responsabilidad bien definida**, fáciles de probar y reutilizar por separado.

Definición de un método

```
[modificadores] tipoDevuelto nombre(parámetros) {
    // cuerpo: lo que hace el método
    return valor;   // solo si tipoDevuelto no es void
}
```

```
public static double calcularIva(double precio) {
    return precio * 0.21;
}
```

- Si el método no devuelve nada, `tipoDevuelto` es `void`.
- Los **modificadores de acceso**, de menos a más restrictivo: `public` (desde cualquier sitio) → `protected` (misma clase, subclases y mismo paquete) → *sin indicar nada* (mismo paquete) → `private` (solo la propia clase). Nunca restringen el acceso desde dentro de la propia clase.
- `static` indica que el método pertenece a la **clase** (se llama como `Math.pow(...)`), no a un objeto concreto — lo retomaremos con mucho más detalle en la unidad 4.

Parámetros: paso por valor, y mutable vs. inmutable

En Java, los parámetros siempre se pasan **por valor**: el método recibe una *copia*. Pero esto significa cosas distintas según el tipo de dato:

- Para tipos **primitivos** (`int`, `double`…), el método recibe una copia del valor — cambiarlo dentro del método **nunca** afecta a la variable original.
- Para tipos **objeto** (arrays, `String`, y las clases que veremos en la unidad 4), el método recibe una copia de la *referencia* — apunta al mismo objeto. Si el objeto es **mutable** (como un array), modificar su contenido dentro del método **sí** se nota fuera. Pero reasignar el parámetro a un objeto distinto dentro del método no cambia la variable original (esa reasignación solo afecta a la copia de la referencia).

```
static void duplicarValores(int[] array) {
    for (int i = 0; i < array.length; i++) {
        array[i] *= 2;   // modifica el array original — es mutable
    }
}

static void intentaCambiar(int x) {
    x = 100;   // no tiene ningún efecto fuera del método
}
```

`String` es un caso especial: aunque es un objeto, es **inmutable** (ninguno de sus métodos cambia el `String` original — `toUpperCase()`, por ejemplo, *devuelve* un `String` nuevo, no modifica el original). Por eso concatenar en un bucle con `String` es ineficiente, y para eso existe `StringBuilder`, que sí es mutable.

Sobrecarga de métodos

Puedes tener varios métodos con el **mismo nombre**, siempre que se puedan distinguir por su número o tipo de parámetros (su *firma*):

```
static int sumar(int a, int b) { return a + b; }
static double sumar(double a, double b) { return a + b; }
static int sumar(int a, int b, int c) { return a + b + c; }
```

Recursividad

Un método puede llamarse a sí mismo. Como en pseudocódigo, necesita siempre un **caso base** que detenga la recursión:

```
static long factorial(int n) {
    if (n <= 1) return 1;              // caso base
    return n * factorial(n - 1);       // caso recursivo
}
```

Precondiciones, postcondiciones y aserciones

Un método bien diseñado documenta (y, si puede, comprueba) lo que espera recibir (**precondición**) y lo que garantiza devolver (**postcondición**). En Java hay dos formas de comprobarlo activamente:

- **Lanzar una excepción** si no se cumple una precondición — la forma habitual en código de producción:

  ```
  static double raizCuadrada(double x) {
      if (x < 0) throw new IllegalArgumentException("x no puede ser negativo");
      return Math.sqrt(x);
  }
  ```

- `assert`: una comprobación pensada para *depuración durante el desarrollo*, que normalmente se desactiva en producción (hay que activarla explícitamente al ejecutar). No sustituye a la validación real de datos de usuario, es una herramienta para que tú, como programador, detectes cuanto antes que una suposición tuya sobre el código era falsa.

  ```
  assert x >= 0 : "x no puede ser negativo";
  ```

5. Excepciones

Java representa los errores en tiempo de ejecución como **objetos**. `Error` (fallos tan graves que el programa no puede recuperarse) y `Exception` (errores de ejecución que sí puedes tratar) son las dos ramas principales bajo `Throwable`.

Atraparlas: `try-catch`

```
try {
    // sentencias que podrían lanzar una excepción
} catch (TipoDeExcepcion variable) {
    // qué hacer si ocurre esa excepción
}
```

```
String texto = "abc";
try {
    double x = Double.parseDouble(texto);
    System.out.println("El número es " + x);
} catch (NumberFormatException e) {
    System.out.println(texto + " no es un número válido: " + e.getMessage());
}
```

Si ninguna sentencia del `try` lanza una excepción, el `catch` simplemente no se ejecuta. Puedes encadenar varios `catch` para tratar distintos tipos de error de forma diferente — **ponlos siempre de más concreto a más general** (si atrapas `Exception` primero, ningún `catch` más específico después de él se llegará a ejecutar nunca):

```
try {
    // ...
} catch (ArithmeticException e) {
    System.out.println("Error aritmético: " + e.getMessage());
} catch (NumberFormatException e) {
    System.out.println("Formato incorrecto: " + e.getMessage());
} catch (Exception e) {
    System.out.println("Otro error: " + e.getMessage());
}
```

Si quieres tratar varios tipos de excepción exactamente igual, puedes unirlos en un único `catch` con `|`:

```
} catch (ArithmeticException | NumberFormatException e) {
    System.out.println("Dato o cálculo incorrecto: " + e.getMessage());
}
```

`finally`: lo que siempre se ejecuta

```java
try {
    int resultado = a / b;
    System.out.println(resultado);
} catch (ArithmeticException e) {
    System.out.println("No se puede dividir entre 0");
} finally {
    System.out.println("Esto se ejecuta siempre, haya o no excepción");
}
```

El bloque `finally` se ejecuta **siempre** — haya habido excepción o no, se haya atrapado o no — y es el lugar habitual para liberar recursos (cerrar ficheros, conexiones…) sin importar cómo haya terminado el `try`.

Lanzar tus propias excepciones: `throw` y `throws`

Puedes lanzar una excepción tú mismo con `throw`, cuando detectes una situación que tu código no debería continuar procesando:

```java
if (edad < 18) {
    throw new IllegalArgumentException("Debe ser mayor de edad");
}
```

Cuando un método puede lanzar una excepción y prefieres no atraparla ahí mismo, avisas de ello en su cabecera con `throws`, delegando la responsabilidad de tratarla a quien lo llame:

```java
public static void procesar(String dato) throws Exception {
    if (dato == null) throw new Exception("Dato nulo");
    // ...
}
```

> **Crear excepciones propias**
>
> Puedes definir tus propios tipos de excepción creando una clase que herede de `Exception` (lo verás con más detalle en la unidad 4, cuando dominemos la herencia). Por ahora, basta con saber que las excepciones “de fábrica” de Java (`NumberFormatException`, `ArithmeticException`, `IllegalArgumentException`…) cubren la inmensa mayoría de los casos habituales.

6. Expresiones regulares

Una **expresión regular** (*regex*) es un patrón de texto que describe el formato que debe tener una cadena — por ejemplo, comprobar que un DNI tiene 8 dígitos seguidos de una letra, o que un email tiene la forma `texto@texto.texto`. Solo comprueba la **sintaxis** (el formato), no el significado (una regex de fecha no sabe que el 31 de abril no existe).

Java te da tres formas de comprobar si un texto cumple una expresión regular, de más simple a más potente:

```
String dni = "12345678A";
String patron = "\\d{8}[A-Z]";   // 8 dígitos + una letra mayúscula

boolean valido1 = dni.matches(patron);                  // opción 1: más simple
boolean valido2 = Pattern.matches(patron, dni);          // opción 2: equivalente

Pattern p = Pattern.compile(patron);                      // opción 3: más potente,
Matcher m = p.matcher(dni);                               // mejor si vas a
reutilizar
boolean valido3 = m.matches();                            // el mismo patrón varias
veces
```

Símbolos básicos que verás constantemente:

Símbolo	Significa
`\d`	Un dígito (0-9)
`\w`	Un carácter “de palabra” (letra, dígito o _)
`[A-Z]`	Una letra mayúscula (rango)
`{n}`	Exactamente `n` repeticiones del elemento anterior
`+`	Una o más repeticiones
`*`	Cero o más repeticiones
`?`	Cero o una repetición (opcional)
`^` / `$`	Principio / final de la cadena

RA y CE que cubre esta unidad

RA	Criterios de evaluación cubiertos
RA1. Reconoce la estructura de un programa informático…	a) estructura de un programa · b) proyectos de desarrollo · c) entornos integrados de desarrollo
RA2. Utiliza estructuras de control y datos…	a) fundamentos POO · b) programas simples · c) instanciación de objetos
RA3. Desarrolla programas estructurados…	a) estructuras de selección · b) estructuras de repetición · c) sentencias de salto · d) control de excepciones · e) programas con distintas estructuras de control · f) probado y depurado · g) comentado y documentado
RA6. Escribe programas que manipulen información con tipos avanzados de datos…	a) programas con arrays · g) expresiones regulares

🟢 Boletín inicial ⭐ Boletín intermedio 🔥 Extras